

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Annexure A

Coding challenge

Total Marks:25

Code of Conduct:

I G.K. Vyshnavi 192425069 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

G.K. Vyshnavi

Coding challenge**1. Star Topology Diagram**

This program takes the number of hosts, switch name, and cable lengths and prints an ASCII star topology.

CODE:

```
n = int(input("Enter number of hosts: "))
```

```
switch = input("Enter switch name: ")
```

```
print("\nSTAR TOPOLOGY")
```

```
print("=" * 40)
```

```
hosts = []
```

```
for i in range(1, n + 1):
```

```
    host = f"Host{i}"
```

```
    length = float(input(f"Enter Cat6 cable length for {host} (m): "))
```

```
    hosts.append((host, length))
```

```
print("\n      " + switch)
```

```
print("      |")
```

```
for host, length in hosts:
```

```
    print(f"    {host} -- {length} m")
```

OUTPUT:

```
Python Visualize Run Output
1 n = int(input("Enter number of hosts: "))
2 switch = input("Enter switch name: ")
3
4 print("***** STAR TOPOLOGY *****")
5 print("-" * 40)
6
7 hosts = []
8
9 for i in range(1, n + 1):
10     host = f"Host({i})"
11     length = float(input(f"Enter Cat6 cable length for {host} (m): "))
12     hosts.append((host, length))
13
14 print("\n          " + switch)
15 print("          |")
          Host1 -- 20.0 m
          Host2 -- 30.0 m
          Host3 -- 40.0 m
          Host4 -- 50.0 m
```

2. UTP Segment Validation

IEEE 802.3 Ethernet twisted-pair links have a 100 m maximum segment length in the common 100 m Ethernet cabling model.

PROGRAM CODE:

```
def validate_segment(length_m):
    if length_m < 0:
        return False, "Error: Length cannot be negative."
    if length_m <= 100:
        return True, "Valid UTP segment."
    return False, "Error: UTP segment cannot exceed 100 m."
```

Test cases

```
tests = [50, 100, 120]
```

for length in tests:

```
    result, message = validate_segment(length)
```



```
print(f"{length} m -> {result} -> {message}")
```

OUTPUT:

main.py	Visualize	Run	Output
<pre>1 def validate_segment(length_m): 2 if length_m < 0: 3 return False, "Error: length cannot be negative." 4 if length_m > 100: 5 return True, "Valid UTP segment." 6 return False, "Error: UTP segment cannot exceed 100 m." 7 8 9 # Test cases 10 tests = [50, 100, 120] 11 12 for length in tests: 13 result, message = validate_segment(length) 14 print(f"{length} m -> {result} -> {message}")</pre>			<pre>ERROR! 50 m -> True -> Valid UTP segment. 100 m -> True -> Valid UTP segment. 120 m -> False -> Error: UTP segment cannot exceed 100 m. === Code Execution Successful ===</pre>

3. CSMA/CD Simulation

This program simulates 4 hosts, detects collisions, calculates a random back-off interval, and stores the events in `csma_log.txt`.

PROGRAM CODE:

```
import random
```

```
import time
```

```
hosts = ["Host1", "Host2", "Host3", "Host4"]
```

```
with open("csma_log.txt", "w") as log:
```

```
    for i in range(6):
```

```
        h1, h2 = random.sample(hosts, 2)
```

```
        log.write(f"{h1} and {h2} are transmitting.\n")
```

```
        log.write("Collision detected!\n")
```

```
        backoff = random.randint(1, 10)
```

```

log.write(f"{h1} back-off: {backoff} ms\n")
log.write(f"{h2} back-off: {backoff + 1} ms\n")
log.write("-" * 30 + "\n")

print(f"Collision: {h1} and {h2}")
print(f"Back-off: {backoff} ms")

time.sleep(0.2)

```

```

print("\nSimulation completed.")
print("Log saved to csma_log.txt")

```

OUTPUT:

Programiz Python Online Compiler

main.py

Visualize

Run

```

1 import random
2
3 hosts = ["Host1", "Host2", "Host3", "Host4"]
4
5 for i in range(5):
6
7     h1, h2 = random.sample(hosts, 2)
8
9     b1 = random.randint(1, 10)
10    b2 = random.randint(1, 10)
11
12    print(h1, "and", h2, "are transmitting")
13    print("Collision detected!")
14    print(h1, "back-off:", b1, "ms")
15    print(h2, "back-off:", b2, "ms")
16    print("-" * 30)
17
18 # Simulation completed

```

Output

```

Host2 and Host4 are transmitting
Collision detected!
Host2 back-off: 9 ms
Host4 back-off: 7 ms
-----
Host3 and Host1 are transmitting
Collision detected!
Host3 back-off: 7 ms
Host1 back-off: 8 ms
-----
Host3 and Host2 are transmitting
Collision detected!
Host3 back-off: 9 ms
Host2 back-off: 4 ms
-----
Host4 and Host1 are transmitting

```

4. Client-Server Star Switch Using Socket

Run the server first, then the client.

The server acts as the central switch and maintains a MAC address table.

PROGRAM:

```
import socket
```

```
import threading
```

```
HOST = "127.0.0.1"
```

```
PORT = 5000
```

```
mac_table = {}
```

```
# ----- STAR SWITCH -----
```

```
def switch():
```

```
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
```

```
    server.bind((HOST, PORT))
```

```
    server.listen(2)
```

```
print("=== STAR SWITCH ===")
```

```
print("Waiting for Host1 and Host2...")
```

```
connections = []
```

```
for i in range(2):
```

```
    conn, addr = server.accept()
```

```
    host = "Host" + str(i + 1)
```

```
    connections.append(conn)
```

```
    mac_table[host] = addr
```

main.py

Visualize

Run

Output

```

77 def host2():
78     while True:
79         if not data:
80             break
81         print("Host2 received:", data.decode())
82     client.close()
83
84
85 # ----- MAIN -----
86 print("Starting Star Network...\n")
87
88 threading.Thread(target=switch, daemon=True).start()
89

```

```

Starting Star Network...

=== STAR SWITCH ===
Waiting for Host1 and Host2...
Host2 connected to switch.
Waiting for frames...
Host1 connected.
Host2 connected.

MAC Address Table:
Host1 -> ('127.0.0.1', 36114)
Host2 -> ('127.0.0.1', 36126)
Host1 connected to switch.
Host1: |

```

5. StarSwitch Class

This program implements:

- add_port()
- remove_port()
- broadcast()
- Delivery logging

program code:

class StarSwitch:

```
def __init__(self):
```

```
    self.ports = []
```

```
def add_port(self, host_id):
```

```
    if host_id not in self.ports:
```

```
        self.ports.append(host_id)
```

```
        print(host_id, "connected.")
```

```
def remove_port(self, host_id):
```



```
if host_id in self.ports:  
    self.ports.remove(host_id)  
    print(host_id, "removed.")
```

```
def broadcast(self, frame):  
    source = frame["source"]  
  
    print("\nBroadcasting frame:", frame["data"])
```

```
for host in self.ports:  
    if host != source:  
        print(f"Frame delivered to {host}")
```

Test

```
switch = StarSwitch()
```

```
switch.add_port("Host1")
```

```
switch.add_port("Host2")
```

```
switch.add_port("Host3")
```

```
switch.add_port("Host4")
```

```
frame = {  
    "source": "Host1",  
    "data": "Hello Network"  
}
```



```
switch.broadcast(frame)
```

```
switch.remove_port("Host4")
```

output:

Programs Python Online Compiler

Visualize Run

Output

main.py

```
1 class StarSwitch:
2
3     def __init__(self):
4         self.ports = []
5
6     def add_port(self, host_id):
7         if host_id not in self.ports:
8             self.ports.append(host_id)
9             print(host_id, "connected.")
10
11     def remove_port(self, host_id):
12         if host_id in self.ports:
13             self.ports.remove(host_id)
14             print(host_id, "removed.")
15
16     def broadcast(self, frame):
17         source = frame["source"]
```

Host1 connected.
Host2 connected.
Host3 connected.
Host4 connected.

Broadcasting frame: Hello Network
Frame delivered to Host2
Frame delivered to Host3
Frame delivered to Host4
Host4 removed.

=== Code Execution Successful ===

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or unclear approach
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written or unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not work for most or all test cases